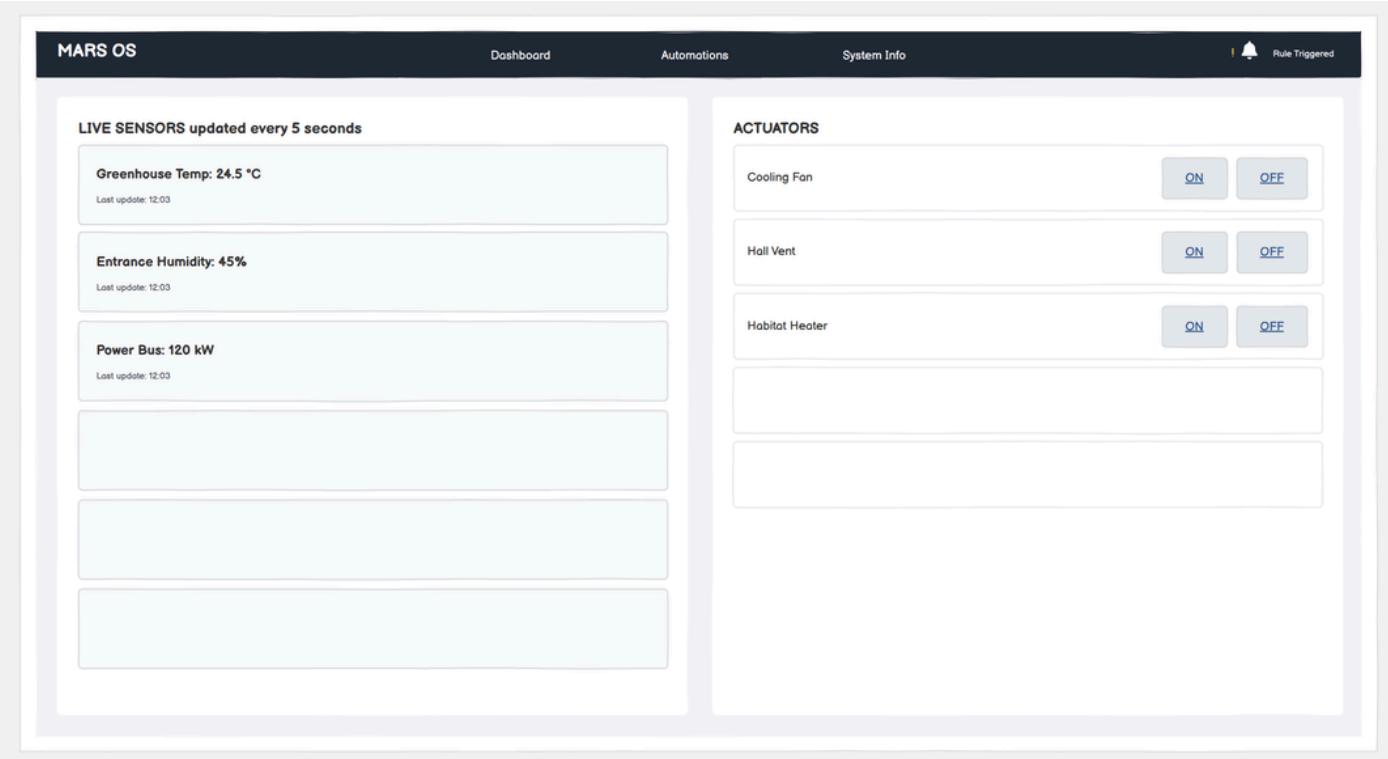
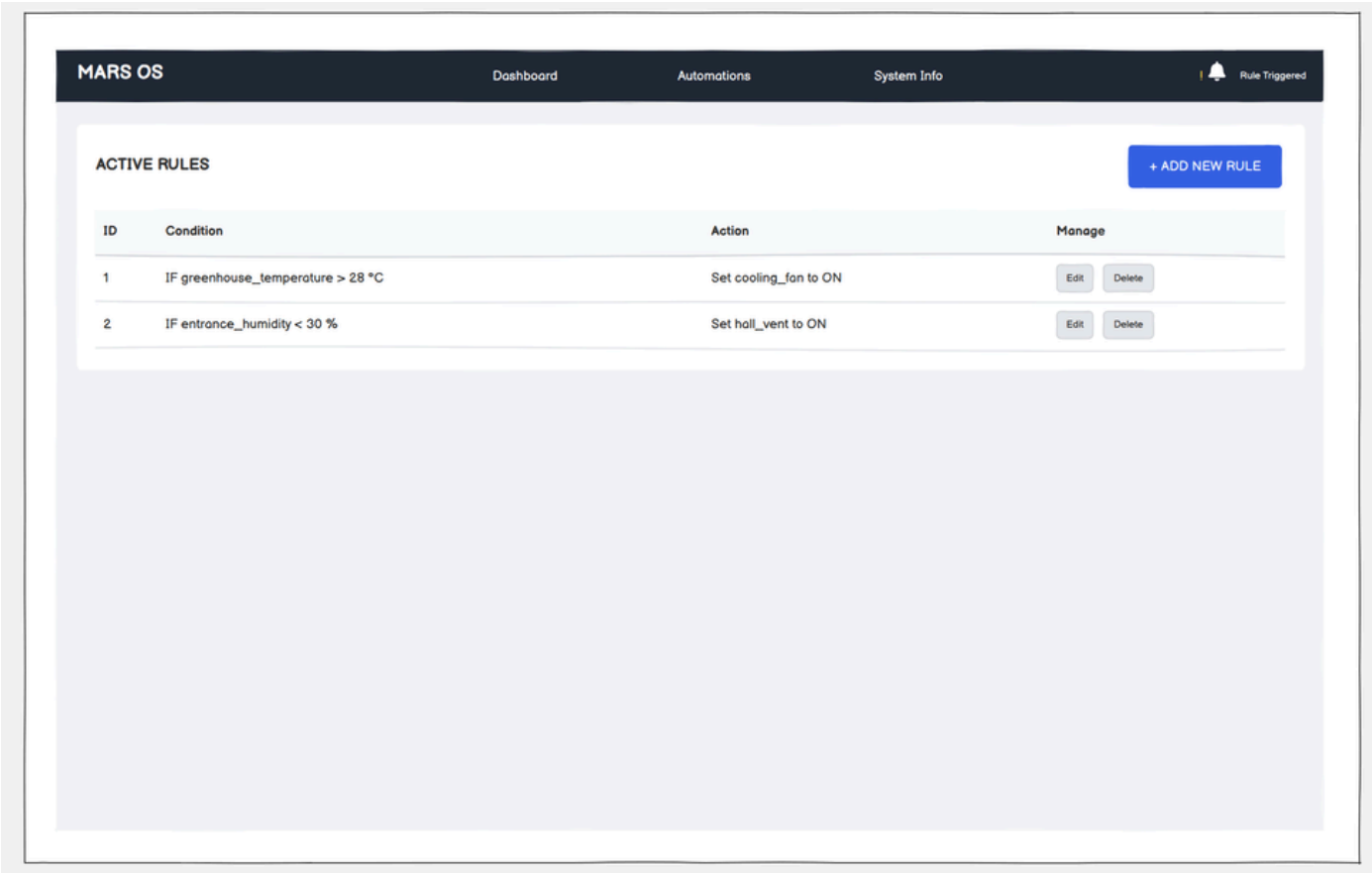
# User Stories + Mockups

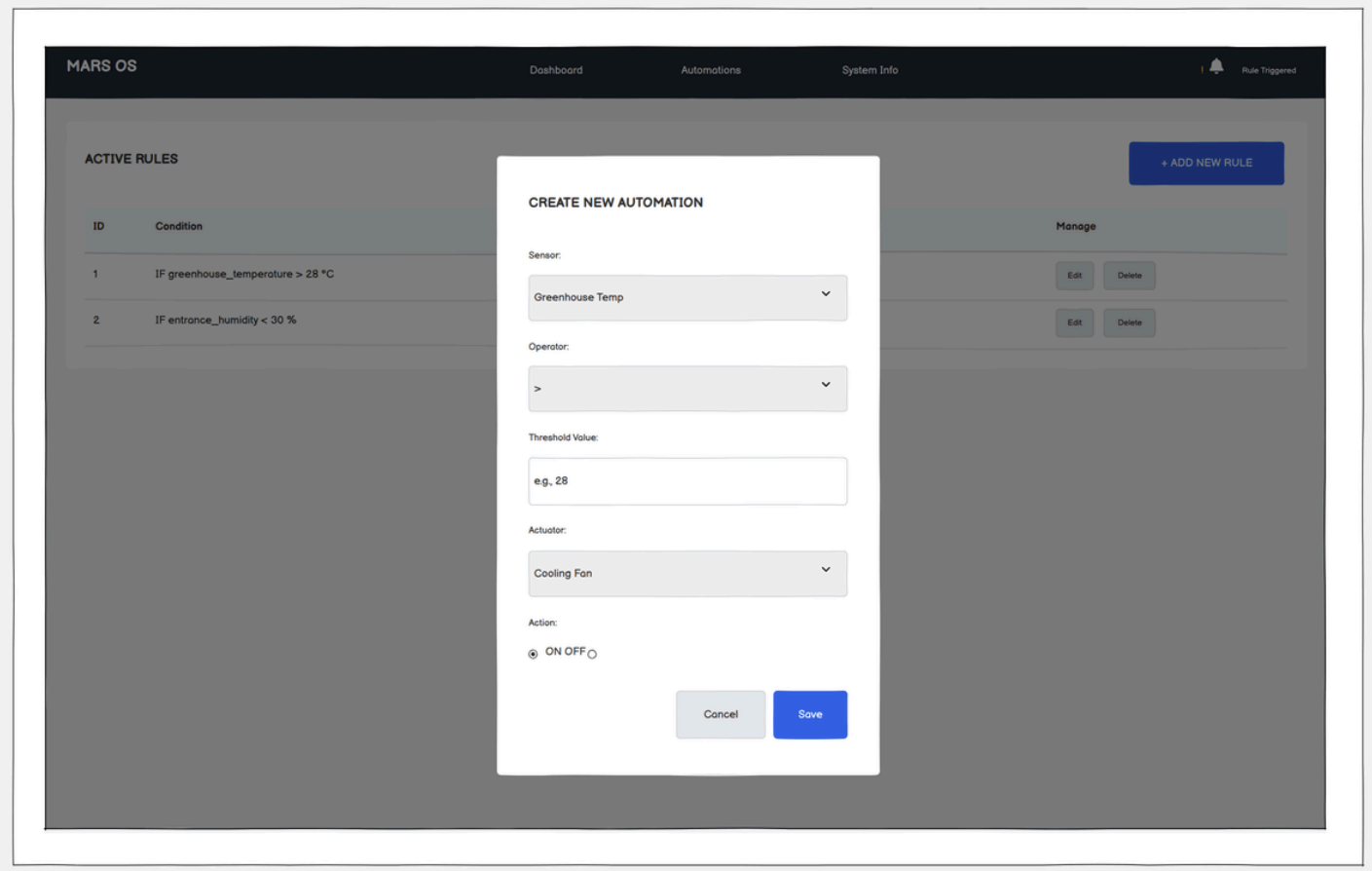
## Mockups related to user stories 1, 2, 3, 4, 5, 9, 19, 20, 21



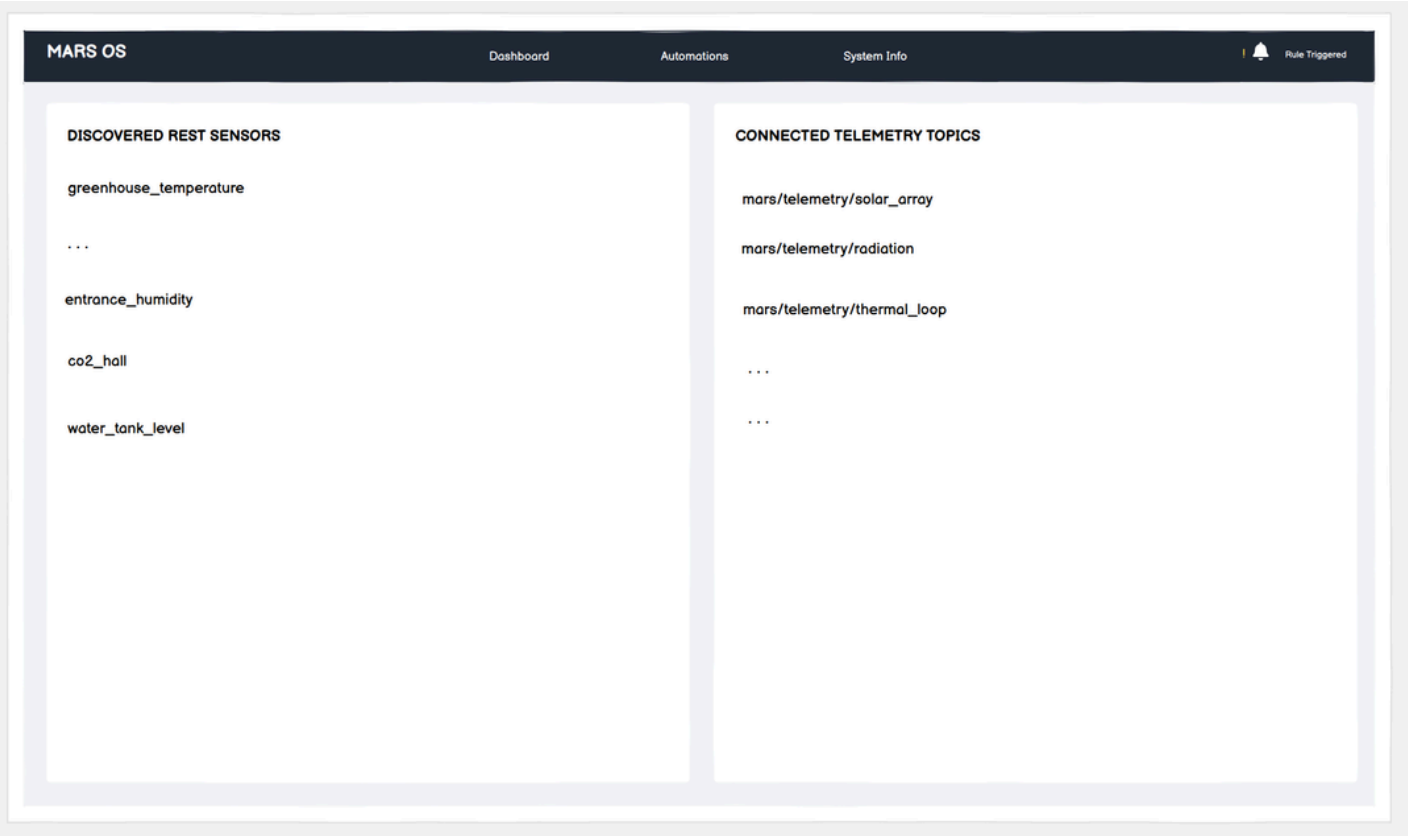

## Mockups related to user stories 10, 15, 16, 17



# Mockups related to user stories 11, 13, 14




# Mockups related to user stories 22, 23



# Description for non functional requirements

- **User Stories 6, 7:** These stories do not have a graphical interface. They represent the backend Ingester Service's functional requirement to continuously poll the /api/sensors endpoints and maintain active connections to /api/telemetry/ws or SSE streams.

- **User story 8:** Architectural requirement. All incoming heterogeneous JSON payloads must be mapped to a unified standard event format before being published to the message broker.

- **User story 12:** Performance/Functional requirement. The Rule Engine must act as a consumer of the message broker, evaluating all active rules in memory the moment a new standardized event arrives, ensuring low-latency reactions.

- **User story 18:** Non-functional requirement (Data Persistence). Rules must be stored in a persistent database so that if the Rule Engine service restarts, the automations are not lost.

- **User story 24:** Architectural constraint. The system must implement an Event-Driven Architecture. Services must not communicate synchronously (except for frontend API calls); instead, they must use a message broker for decoupled communication.

- **User story 25:** Deployment constraint. The entire platform (Broker, DB, Backend services, Frontend, and the Simulator) must be containerized and orchestrated via a single docker-compose.yml file, starting automatically without manual intervention.